

Homework Five : More on Lists

Question One

A deque (pronounced deck) is an ordered set of items from which items may be deleted at either end and into which items may be inserted at either end. Call the two ends left and right. This is an access-restricted structure since no insertions or deletions can happen other than at the ends. Implement the deque as a doubly-linked list (not circular, no header). Write InsertLeft and DeleteRight

Deque as a doubly linked list

For the deque, we need a doubly linked list. This will consist of nodes hav data, prev, and next. The deque will just have a head and tail pointer.

```
typedef struct {
    data;
    node_t prev; // pointers
    node_t next;
} node_t;

typedef struct {
    node_t left;
    node_t right;
} deque_t;

class deque;
// class members
deque_t my_deque;

// class methods
void insertLeft(data) {
    node_t new_node;
    new_node.prev = null;
    new_node.next = my_deque.left;

    if(my_deque.left == null)
        my_deque.right = ne_node;
    else
        my_deque.left.prev = new_node

    dq.left = new_node;
}

void deleteRight {

    if my_deque.right == null
        $error

    else
```

```

        my_deque.right = my_deque.right.prev;
        my_deque.right.next = null;
    }

    endclass: deque

```

Question Two

Implement a deque from problem 1 as a doubly-linked circular list with a header. Write InsertRight and DeleteLeft.

```

typedef struct {
    data;
    node_t next;
    node_t prev;
} node_t;

typedef struct {
    node_t header;
} deque_t;

class deque;
    // Class Member
    deque_t my_deque;

    // Class methods
    void new {
        my_deque.header.prev = my_deque.header;
        my_deque.header.next = my_deque.header;
    }

    void insert_right(data) {
        node_t new_node;
        new_node.data = data;
        new_node.next = my_deque.header;
        new_node.prev = my_deque.header.prev;
        my_deque.header.prev = new_node;
    }

    void delete_left(data) {
        if my_deque.header.next == deque.header
            $error (can't delete, only header)
        else
            my_deque.header.next = my_deque.header.next.next
    }

```

```
endclass: deque
```

Question Three

Write a set of routines for implementing several stacks and queues within a single array. Hint: Look at the lecture material on the hybrid implementation.

So, in order to approach this problem, I found it easiest to use a node-based array, where the array is contiguous but the data structures aren't.

Thus, each element will need to have a next pointer, such that the stack in reverse order.

```
typedef struct { next; val; } node_t;
node_t big_array[100];
int free_list[100] = {0};
int stack_top[n] = {-1};
int queue_fronts[n] = {-1};
int queue_backs[n] = {-1};

// Memory Managment Stuff
function allocate()
    if(all_ones(free_list)) $display("Memory full");
    else return find_first_zero(free_list);
endfunction: allocate

function freee(index)
    free_list[index] = 0;
endfunction: freee

// Stack Operations
function stack_push(stack_id, val)
    int index = allocate();
    if(index == -1) $display("Stack Overlfow");
    else {
        big_array[index].val = val;
        big_array[index].next = stack_top[stack_id];
        stack_top[stack_id] = index;
        free_list[index] = 1;
    }
endfunction: stack_push

function stack_pop(stack_id)
    if(stack_top[stack_id] == -1) $display("Stack Underflow");
    else {
        int index = stack_top[stack_id];
        int val = big_array[index].val;
        stack_top[stack_id] = big_array[index].next;
        freee(index);
        return val;
    }
}
```

```
endfunction: stack_pop

// Queue Operations
function enqueue(queue_id, val)
    int index = allocate();
    if(index == -1) $display("Queue Overlfow");
    else {
        big_array[index].val = val;
        big_array[index].next = -1;
        if(queue_backs[queue_id] == -1) queue_fronts[queue_id] = index;
        else big_array[queue_backs[queue_id]].next = index;
        queue_backs[queue_id] = index;
        free_list[index] = 1;
    }
endfunction: enqueue

function dequeue(queue_id)
    if(queue_fronts[queue_id] == -1) $display("Queue Underlfow");
    else {
        int index = queue_fronts[queue_id];
        int val = big_array[index].val;
        queue_fronts[queue_id] = big_array[index].next;
        if(queue_fronts[queue_id] == -1) queue_backs[queue_id] = -1;
        freee(index);
        return val;
    }
endfunction: dequeue
```